

USER DOCUMENTATION · REVISION 2026.01

Spine Skeleton Shader 使用說明書。

本文件供工作室美術、特效與程式同仁參閱。涵蓋 Shader、Inspector、EditorWindow、Runtime 元件、Preset 預設檔、打包設定，以及所有右鍵選單 / 頂部說明按鈕背後的指南視窗。

§ 1–5 導讀：架構、Inspector、Help 系統

§ 17–24 Preset 預設檔系統 + 程式 API

§ 6–11 Shader 六大分區全部參數

§ 25–28 打包流程 · 平台變體中心

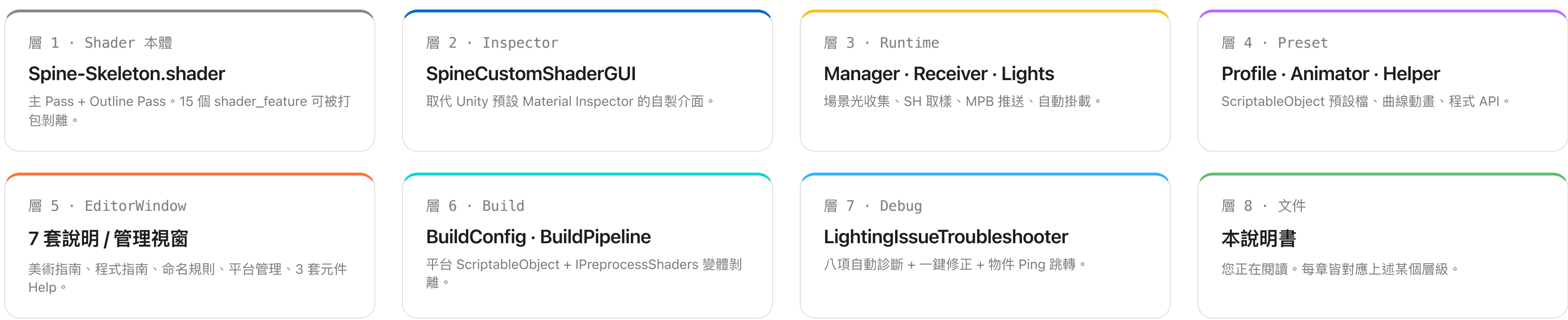
§ 12–16 元件 / 自動掛載 / Runtime

§ 29–33 Layer 規約 · 排查器 · Quick Start

§ 1 · 系統組成

本系統由四層結構組成。

使用者僅需操作第 2 層 Inspector 與第 5 層 EditorWindow。其餘各層為自動運作 — 在勾選功能時自動掛載、注入資料、並於打包時依平台設定剝離不必要的變體。



§ 2 · GUI 分區總覽

Inspector 由六個可摺疊分區構成。

點擊任一分區色帶可摺疊 / 展開該區。右鍵分區色帶可「複製整區設定 / 貼上 / 重置」。每個子模組標題（如 Native Lighting Settings）右側皆有 P 圖示，展開後即為該子模組的 Preset 面板。詳細參數見 §6–§11。

§ 6 · MAIN

Main & Shadow / Post-Processing

主紋理、Tiling/Offset、主色、Cutoff、Straight Alpha、Bloom 抑制門檻同步。

§ 7 · NOISE

Global Noise Settings

UV 動態雜訊 — 提供 BackLight / Inline / Outline 共用的隨機遮罩。

§ 8 · LIGHTING

Native · ShadowLight Base · NormalMap · Probe

Unity Realtime 燈、自製 ShadowLight、法線貼圖、Light Probe SH。

§ 9 · EMISSION

Emission Settings

獨立 Emission 貼圖 + HDR 顏色 + Multiply with MainTex 選項。

§ 10 · EFFECT

Overlay · Hit Sweep · BackLight

整體染色、受擊光帶、背光高光 — 三組可獨立啟用。

§ 11 · LINE

Inline · Outline

內描邊（Rim 風格）、外描邊（Sobel 8 鄰域）— 共用 Light Source 結構。

§ 3 · INSPECTOR 視覺符號

每個符號都有固定含意。

本節列出 Inspector 中會反覆出現的五項視覺符號與其行為。理解這些符號後，可以從外觀立刻判斷每個欄位的狀態與可操作項目。

3px 上色帶	分區頂部 — 點擊摺疊整區；右鍵彈出 Copy / Paste / Reset 選單
3px 左色條	子模組標題（如 Native Lighting Settings）
+	該屬性當前正受某 Preset 驅動，數值被覆寫
~	將屬性切到 AnimationCurve 動畫驅動（按一次再展開曲線編輯區）
P	開關該子模組的 Preset 面板（Profile 欄位 + Load / Save / Clear）
■	頂部「使用指南」按鈕 — 開啟 EditorWindow 顯示完整文件



圖示：實際 Inspector 顯示樣式（簡化示意）

§ 4 · 兩個入口 — 找到任一屬性的說明

每個參數都能立刻查到完整文件。

所有自製 Inspector（Material GUI、ShadowLight、HitSweep、Profile）皆內建統一的查詢介面。當你不確定某個欄位的用途、依賴條件或排錯方式時，請使用以下兩個入口：

① 右鍵任一屬性

對 Inspector 上任何一個屬性欄位按下滑鼠右鍵，會跳出 GenericMenu，選單中含「 查看使用說明」項目。

點擊後系統會：

- 開啟對應的 EditorWindow（美術指南 / 元件 Help）
- 自動展開包含該屬性的章節
- 自動捲動到該屬性卡片並以 Ping 黃色閃爍 2 秒

② 頂部 使用指南按鈕

每個 Inspector 頂部都會繪製一條紅色 / 藍色 / 黃色橫條，標題為「 XXX 使用指南」。點擊可開啟對應的完整 EditorWindow。

每個 EditorWindow 都包含：

- 頂部搜尋框 — 可搜尋屬性名 / 關鍵字
- 可摺疊的模組分區（按色條展開）
- 每個屬性附「依賴條件」與「排錯檢查」標籤

設計理念 — 任何在 Inspector 看到的欄位都應該能在 30 秒內找到對應說明。若某個屬性沒有右鍵說明或頂部按鈕無法開啟，請通報維護者。

§ 5 · 七個 EDITORWINDOW 一覽

說明文件本身也是工具的一部分。

系統內建七個 EditorWindow — 五個是「說明指南」、一個是「平台管理」、一個是「Profile 編輯器」。下表列出全部入口、用途與本書的對應章節。

SpineArtHelpWindow	美術視覺開發指南 — 每個 Shader 屬性的友善名稱、用途、依賴、排錯說明。Material Inspector 頂部按鈕可開啟。	■ 美術使用指南 → §6–§11
SpineDebugHelpWindow	常見問題排錯 Q&A：角色為何全黑、勾選效果為何無效。整合 SpineLightingIssueTroubleshooter 排查面板。	■ 除錯指南 → §30
SpineProfileHelpWindow	程式調用 + MPB 衝突防護指南。含 ApplyProfileRuntime、ApplyTempProfile、SetFloat/Color/Vector 程式碼範例。	■ 程式 API 指南 → §22–§24
SpineNamingRuleWindow	Preset 預設檔命名規則 — 每個模組附通用型與角色特化型範例 + 一鍵複製名稱。	■ 命名規則 → §21
SpineShadowLightHelpWindow	SpineShadowLight 元件指南 — Light Settings 與 Cookie Settings 全部欄位說明。最多 8 盞同時影響角色。	■ 場景燈光指南 → §12
SpineHitSweepHelpWindow	Hit Sweep 動畫設定指南 + PlaySweep(Vector3) 程式 API 範例與一鍵複製。	■ 掃光指南 → §13
SpinePlatformManagerWindow	平台變體管理中心 — 滑動切換不同平台的 Shader Feature 啟用組合、即時刷新全專案材質。	Window 選單入口 → §25–§28

§ 5.1 · 通用 HELP WINDOW 介面

搜尋、摺疊、Ping 高亮 — 七套 Window 一致行為。

所有 Help Window 採同一個 UI 範本。除了內容不同，操作邏輯完全一致。學一次即可。

頂部搜尋	即時過濾屬性名稱、友善名稱、描述、解決方案文本。
模組色帶	點擊整條色帶可摺疊 / 展開該模組。搜尋時自動展開有命中的模組。
屬性卡片	標題顯示友善名稱 + ()內顯示 Shader 變數名稱 + 描述 + [依賴條件] + [排錯檢查]。
Ping 高亮	由右鍵跳轉時，目標卡片會被黃色矩形閃爍 2 秒以利視覺定位。
複製按鈕	程式 API 區的程式碼方塊可一鍵複製到剪貼簿。

🔍 輸入關鍵字搜尋效果或屬性...

Spine Shader 美術視覺開發指南

◆ Lighting Settings (光照系統)

◆ Master Light Intensity (_NativeLightIntensity)

全域控制所有 Unity 燈光對角色的影響強度。

依賴 需開啟 Enable Native Lighting。

◆ Enable Light Probe (_EnableLightProbe)

讓角色受到烘焙環境光影響。場景開啟 LightMap + LightProbe 時自動開啟。

排錯 若按鈕一直跳回關閉，代表場景缺乏 Lightmap 或 Probe 資料。

◆ Effect Overlay & BackLight

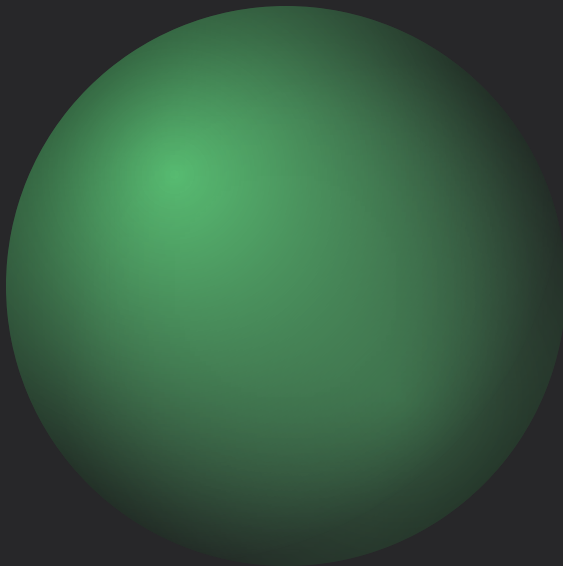
◆ Tint By Light Color (_BackLightTintByLight)

勾選後，背光顏色會與場景燈光顏色混合。

主紋理與基礎參數。

本章說明 MAIN 分區所含兩個子模組（Main & Shadow / Post-Processing Control）的所有屬性。本分區為所有材質必要設定，無法停用。

章節範圍	Main & Shadow Settings · Post-Processing Control
自動掛載	Suppress Bloom 開啟 → 自動掛載 SpineBloomThresholdSync
前置條件	無



§ 6.1 · MAIN

Main & Shadow Settings

系統會自動偵測同名後綴貼圖（_Normal / _BackLight / _Emission）並掛載。將材質拖到 SpriteRenderer 時亦會自動同步 MainTex 與 Straight Alpha 設定。

Main Texture	主紋理。拖材質到 SpriteRenderer 時系統會自動讀取 Sprite 貼圖並開啟 Straight Alpha。	_MainTex
Shared Tiling & Offset	共用 UV (XY=Tiling, ZW=Offset)。此值會同步影響 MainTex、Emission、BackLight、Normal Map 確保所有疊加層對齊。	_MainTex_ST
Main Color (RGBA)	基礎顏色與全域透明度。Alpha 若小於 1 會連同 BackLight / Inline / Outline 一起變透明。Alpha 為 0 時若出現鬼影，多為 Beautify 壓暗或 Straight Alpha 未勾選所致。	_Color
Shadow Alpha Cutoff	Shadow Pass 的 Alpha 裁切閾值；小於此值的像素不投射陰影。	_Cutoff · 0–1
Straight Alpha Texture	切換貼圖透明度計算模式。Spine 預設為 PMA；若邊緣出現黑邊或白邊請切換此項。	_StraightAlphaInput

自動偵測 — 若主紋理位於與 _Normal / _BackLight / _Emission 同目錄且檔名遵循「主貼圖名稱_後綴」格式（例如 Char_A.png 與 Char_A_Normal.png），系統會在材質選取時自動填入。

§ 6.2 · MAIN

Post-Processing Control — 後製 Bloom 抑制

後製管線（如 WinkingBeautify）的 Bloom 經常將 HDR 描邊與發光過度過曝。本模組讓 Shader 取得目前後製 Bloom 強度，並依 AnimationCurve 反向換算出本材質專屬的抑制門檻。

Suppress Bloom	啟用後系統自動掛載 SpineBloomThresholdSync 元件至角色。	<code>_EnableBloomSuppression</code> → Layer 必須正確
Bloom Threshold	當前計算出的抑制門檻 — 由 SpineBloomThresholdSync 自動填入，唯讀，不需手動修改。	<code>_BloomSuppressThreshold</code>

SpineBloomThresholdSync 預設曲線	
起點	Bloom = 1.0 → Threshold = 0.6000
終點	Bloom = 10.0 → Threshold = 0.5209
資料來源	Reflection 抓取 WinkingBeautify.finalBloomIntensity
寫入方式	MaterialPropertyBlock（不污染材質球）
效能策略	髒標記（lastAppliedBloom）— 數值未變則跳過寫入

排錯 — 若 Suppress Bloom 勾選後仍過曝，請確認角色 Layer ∈ {8, 9, 12, 13, 18, 20}。Layer 不正確時自動化機制會將 BloomThresholdSync 元件移除。

Global Noise Settings — 全域雜訊取樣。

本模組不直接影響畫面 — 它是供 BackLight、Inline、Outline 等模組選用的「亂度遮罩」來源。三者讀取同一張動態雜訊，因此節奏、速度、方向保持一致。當其他模組啟用 Noise Mask 時，本分區自動連動開啟。

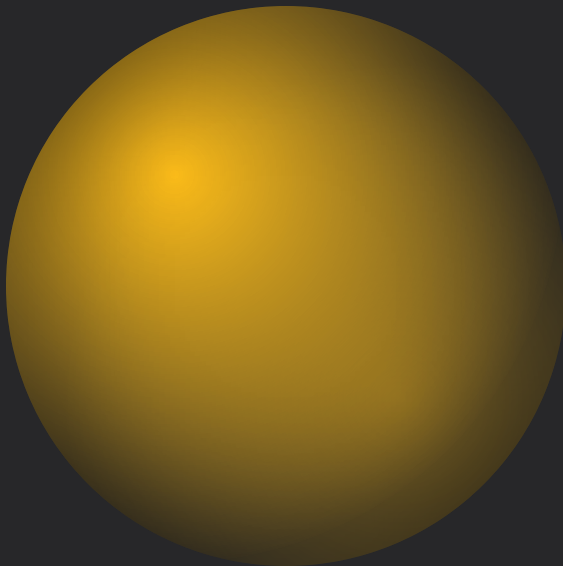
Enable Global Noise	啟用 Shader 全域噪點運算，可用於消融、火焰外框等動態效果。	<code>_EnableGlobalNoise</code> → 啟用後需於子模組勾選 <code>Noise Mask</code>
Noise Scale (X, Y)	UV 平鋪倍率。值越大顆粒越細密；建議 X 與 Y 同步調整。	<code>_GlobalNoiseScale</code>
Noise Speed (X, Y)	UV 流動速度。若感覺沒在流動，請將此值調大。	<code>_GlobalNoiseSpeed</code>

設計守則 — Noise 為共用資源。若你在多個模組同時啟用 Noise，預期它們會同節奏變化；需要差異感請改用各模組獨立的 Intensity 調幅。

光照系統 — 四個子模組。

本章依序介紹 Native Lighting（Unity Realtime）、Shadow Light Base（自製 SpineShadowLight）、Normal Map（法線取樣）、Light Probe（環境光 SH 取樣）。後三者皆建立於 Native Lighting 的資料流之上，由 SpineLightManager 集中處理。

章節範圍	4 子模組 · 共 14 個屬性
自動掛載	啟用任一子模組 → 自動掛載 SpineLightReceiver；EnsureInstance 建立 LightManager
前置條件	角色 Layer $\in \{8, 9, 12, 13, 18, 20\}$



§ 8.1 · LIGHTING

Native Lighting Settings

接收 Unity 場景的 Realtime Directional / Point / Spot Light。所有燈光資料由 SpineLightManager 依距離排序取前 8 盞，透過 MaterialPropertyBlock 推送 — 不需手動寫腳本。

Enable Native Lighting	啟用接收 Unity Realtime Light；關閉後本模組所有屬性失效。	<code>_EnableNativeLighting</code> → 角色全黑時請確認 <code>Master Intensity</code> 不為 <code>0</code>
Master Light Intensity	所有 Realtime 光的總強度倍率 — 用於整體角色亮度調節。	<code>_NativeLightIntensity</code> · <code>0-5</code>
Directional Intensity	場景中 Directional Light（太陽光）的獨立倍率。	<code>_DirectionalLightIntensity</code>
Additional Intensity	場景中 Point / Spot Light 等附加光源的倍率，與 Directional 分開控制。	<code>_AdditionalLightIntensity</code>
Light Affects Additive	是否讓光照影響 Spine Additive Slot；預設關閉避免粒子過曝。	<code>_LightAffectsAdditive</code>
Probe → Native Lighting	Light Probe 取樣值疊加到 Native Lighting 結果的比重；需先啟用 Light Probe。讓暗部不至於死黑。	<code>_LightProbeNativeBlend</code> · <code>0-1</code> → 需 <code>Enable Light Probe</code>

§ 8.2 · SHADOW LIGHT BASE

Shadow Light Base Settings

接收所有 SpineShadowLight 元件投射的自製光照 — 區域光、聚光、Cookie 投影、Caustics 波光。

Enable Shadow Light Base	影響 BaseColor 的基底光照層（不含描邊與背光）。	_EnableShadowLight
Base Intensity	ShadowLight 在 BaseColor 上的整體強度乘數。	_ShadowLightBaseIntensity · 0–5
Affects Normal Map	是否參與 Normal Map 計算 — 開啟後光源方向會推算法線著色。	_SL_AffectsNormalMap → 需開啟 Normal Map

§ 8.3 · NORMAL MAP

Normal Map Settings

讓 2D Sprite 接收方向性光照。自動偵測同目錄的 _Normal 後綴貼圖並掛載。

Enable Normal Map Module	啟用法線採樣與光照計算。	_EnableNormalMap
Normal Map	法線貼圖。若主紋理同目錄有 {name}_Normal，會自動填入。	_NormalMap
Invert Normal Height	反轉法線 Z 軸高度（凹凸方向）。	_InvertNormalMap
Normal Intensity	法線強度。0 為平面、1 為原始、5 為極度誇張。	_NormalIntensity · 0–5

協同運作 — Normal Map 啟用後會被 Native Lighting、Light Probe、ShadowLight（若 Affects Normal Map = ON）三者同時取用。一張法線貼圖即可撐起整個立體感。

§ 8.4 · LIGHT PROBE

Light Probe Settings

取樣場景烘焙好的 Light Probe Group，把 9 個 SH 係數傳入 Shader。若場景開啟 LightMap + LightProbe 時此選項自動開啟。

Enable Light Probe	總開關。BackLight / Inline / Outline 若選 LightProbe 光源，本選項必須啟用。若按鈕一直跳回關閉，代表場景缺乏 Lightmap / Probe 資料。	<code>_EnableLightProbe</code>
Probe Intensity	SH 採樣的整體強度倍率。	<code>_LightProbeIntensity</code> · 0–3

進階區塊三件套 — 當 Light Source ≠ Disabled 時，BackLight / Inline / Outline 模組會額外展開三組進階區塊：**A. Mask By Light Direction**（方向遮罩，三光源各有 Power 指數）；**B. Tint By Light Color**（光源染色，三光源各有 Blend + Intensity）；**C. Noise Mask**（噪點侵蝕，需 Global Noise 啟用）。三個模組結構完全相同。

§ 8.5 · LIGHT SOURCE 多選旗標

三種光源 · 共用結構

BackLight、Inline、Outline 三個模組共用一套 Light Source 多選欄位（EnumFlagsField）。每勾選一個光源，下方展開該光源獨立的 Power / Color Blend / Intensity 三組旋鈕。

Flag 1 Realtime Unity 場景燈，前 8 盞	Flag 2 LightProbe 烘焙環境光 SH	Flag 4 ShadowLight 自製 SpineShadowLight
---------------------------------------	----------------------------------	--

對應 Shader 變數：`_BackLightLightingMode` · `_InlineLightingMode` · `_OutlineLightingMode`（整數位元欄位）

§ 9 · EMISSION

Emission Settings — 自體發光

自動偵測 {name}_Emission 後綴貼圖。Emission 不接收任何光源 — 用於符文、機甲、武器尖端、Magic 物件等需要持續發光的區域。

Enable Emission	啟用發光通道；關閉後完全跳過該段 Shader 計算。	_EnableEmission
Emission Texture	指定發光部位遮罩 (黑白或彩色)。自動偵測同目錄 {主貼圖名稱}_Emission。	_EmissionTex
Emission Color (HDR)	疊加的發光顏色 — HDR 強度大於 1 會觸發後製 Bloom 過曝。	_EmissionColor
Blend Weight	發光與底色的混合權重 (0 = 不顯示，1 = 完全替換)。	_EmissionBlend · 0–1
Intensity	發光整體強度倍率，0–10。配合 HDR Color 推 Bloom。	_EmissionIntensity · 0–10
Multiply With MainTex	開啟後發光被主紋理乘調 — 讓符文受角色顏色影響；關閉則完全獨立。	_EmissionMulMain



三個獨立特效模組。

EFFECT 分區包含三個彼此獨立的特效模組：Effect Overlay（瞬間整體染色）、Hit Sweep（受擊光帶掃光）、BackLight（背光高光）。前兩者為時間性特效、適合配合動畫；BackLight 為常駐光照特效、結構最為複雜。

章節範圍	3 子模組
自動掛載	Hit Sweep → SpineHitSweepEffect 元件
前置條件	BackLight 建議搭配 §8 LIGHTING 已配置完成

§ 10.1 · EFFECT OVERLAY

整體色彩疊加

將指定 HDR 顏色按 Intensity 線性疊到最終畫面色。瞬間性特效 — 用於受擊閃白、變身、無敵、燃燒、冰凍。

Effect Color (HDR)	受擊閃白用 (5,5,5)；火焰用 (3,1.2,0.4)。 → 若由程式 Runtime 控制，Preset 請取消勾選此項	_EffectColor
Effect Intensity	0–1 疊加比例。常配合 Curve 做 0→1→0 三幀閃光。	_EffectIntensity

§ 10.2 · EFFECT

Hit Sweep Settings — 受擊掃光特效

「打擊閃光」特效。傳入受擊點世界座標後，光帶從受擊點向外掃過整個角色。常用於武器命中、技能擊發、無敵讀條的視覺反饋。Shader 屬性在 Material Inspector 設定外觀；觸發則由 SpineHitSweepEffect 元件腳本控制（詳見 §13）。

Enable Hit Sweep	啟用打擊光帶；同時系統自動掛載 SpineHitSweepEffect 至角色。	<code>_EnableHitSweep</code> → 角色 Layer 須正確
Sweep Color (HDR)	光帶顏色。預設 HDR 白光 (2, 2, 2) 觸發 Bloom 過曝產生亮邊。	<code>_SweepColor</code>
Sweep Width	光帶寬度，以角色直徑比例計算。0.15 為標準，0.3 較粗。	<code>_SweepWidth</code> · 0.01–1
Sweep Softness	光帶邊緣的羽化柔和度，數值越大邊緣越糊。	<code>_SweepSoftness</code> · 0.001–1

進階控制屬性（位於 SpineHitSweepEffect 元件 Inspector 而非 Material） — `duration`（掃光總時間，秒）、`sweepRange`（Shader 中光帶起點到終點的座標範圍，預設 -4 → 4）、`sweepCenterOffset`（角色中心校正向量，因 Spine 原點通常在腳底）。詳見 §13。

§ 10.3 · EFFECT

BackLight Settings — 基底參數

背光 / 高光通道。BackLight 是 EFFECT 分區中最複雜的模組 — 接受三種光源並產生獨立的 Power / Blend / Intensity 旋鈕集合。自動偵測 {name}_BackLight 後綴貼圖並掛載。

Enable BackLight	總開關。啟用後出現 Light Source 多選欄位與下方進階區塊。	_EnableBackLight
Light Source	EnumFlags — Realtime / LightProbe / ShadowLight 多選；每選一個展開該光源的 Power / Blend / Intensity 三組旋鈕。	_BackLightLightingMode
Ignore Behind Fade (Shadow Light)	當 Shadow Light 位於角色背後時忽略漸隱衰減。常用於「全方向光罩」效果。 → Light Source 需包含 ShadowLight	_BL_SL_IgnoreBehind
BackLight Texture	背光紋理 — 通常是手繪的高光遮罩。預設為純白。自動偵測 {name}_BackLight。	_SpecularTex
Specular Color (HDR)	背光顏色 (HDR)。冷色銀邊用 (0.6, 0.9, 1.5)；火焰邊用 (1.5, 0.8, 0.3)。	_SpecularColor
Blend Intensity	背光疊加比例 0–1。	_SpecularBlend
Blend Mode	Replace（蓋上去）或 Additive（加上去）。	_BackLightMode
Multiply With MainTex	背光是否受主紋理顏色影響。開啟讓邊緣高光更貼著角色。	_SpecularMulMain

§ 10.3 · EFFECT

BackLight Settings — 進階控制（與 Inline / Outline 共用結構）

當 Light Source 不為 Disabled 時，會展開三個進階區塊。本頁說明的結構同時適用於 Inline (§11.1) 與 Outline (§11.2) — 一次學會三處受用。

A · Mask By Light Direction

方向遮罩

開啟後背光只顯示在受光面。為三種光源各提供 Power 指數，控制聚焦銳利度。

_BL_RT_Power

_BL_Probe_Power

_BL_Shadow_Power

B · Tint By Light Color

光源染色

開啟後 BackLight 顏色被光源色染色。每個光源獨立的 Blend 與 Intensity 控制深度。

_BL_RT_Blend / _Intensity

_BL_Probe_Blend / _Intensity

_BL_Shadow_Blend / _Intensity

C · Noise Mask

流動噪點

開啟後背光受 Global Noise 調幅 — 邊緣像火焰般顫動。需啟用 §7 Global Noise。

_BL_EnableNoise

_BL_NoiseIntensity

對應命名規則 — Inline 模組的進階屬性以 _IL_ 前綴（_IL_RT_Power、_IL_RT_Blend …）；Outline 模組以 _OL_ 前綴。命名規律一致。

兩種描邊 — 向內與向外。

Inline 從角色 Alpha 邊緣向內取樣漸層（Rim Light 風格）；Outline 在外側以 Sobel 8 鄰域偵測繪製外輪廓。兩者共用同一套 Light Source 多選 + Directional Mask / Tint By Light / Noise Mask 結構（§10.3 進階區塊）。

章節範圍	2 子模組 + 進階屬性
自動掛載	Inline → SpineAlphaMaskRenderer
前置條件	Outline 為獨立 Pass，可被打包剝離

§ 11.1 · INLINE (RIM LIGHT)

Inline Settings

內描邊。啟用後系統自動掛載 SpineAlphaMaskRenderer，離線渲染一張 Alpha Mask RT 供 Shader 取樣（§16）。

Enable Inline	啟用內描邊；自動掛載 AlphaMaskRenderer。	<code>_EnableInline</code> → Layer 須正確
Light Source	同 BackLight 結構。可全停用做純色內描邊。	<code>_InlineLightingMode</code>
Inline Color (HDR)	內描邊顏色，HDR。	<code>_InlineColor</code>
Inline Width	描邊寬度（像素）。	<code>_InlineWidth</code> · 0–10
Fade Steps	內描邊階梯數 2–8。值越大邊緣越平滑。	<code>_InlineFadeSteps</code>
Inline ZTest	LessEqual / Always；後者可穿透前景顯示。	<code>_InlineZTest</code>

進階區塊（Directional Mask / Tint / Noise）以 `_IL_` 前綴，結構同 §10.3。

§ 11.2 · LINE

Outline Settings — 基底

外輪廓描邊。獨立 Pass 「Outline」 以 Sobel 邊緣偵測在 Alpha 邊界外側繪製描邊。支援被 Stencil 標記、被光源染色、被深度測試覆蓋等策略。

Enable Outline	啟用外描邊；Pass 「Outline」 會被啟動，需在 Build Config 中放行。	_EnableOutline
Light Source	同 BackLight / Inline 結構，三光源多選。	_OutlineLightingMode
Ignore Behind Fade (Shadow Light)	Shadow Light 位於角色背後時忽略漸隱衰減。	_OL_SL_IgnoreBehind
Outline Color (HDR)	外描邊顏色，HDR 過曝可推 Bloom 形成光暈邊。	_OutlineColor
Outline Width	外描邊寬度，0–8 像素或螢幕空間單位。	_OutlineWidth
Multiply Edge Color	將外描邊與主紋理邊緣顏色相乘 — 讓邊緣繼承角色本色。	_MultiplyEdgeColor
Outline Smoothness	邊緣柔和度 (Threshold End)，控制 Alpha 漸變開始位置。	_ThresholdEnd · 0–1
Alpha Cutoff	取樣的 Alpha 裁切閾值。	_OutlineAlphaCutoff · 0–1
Outline ZTest	LessEqual / Always；後者可穿透前景物件顯示。	_OutlineZTest

§ 11.2 · LINE

Outline · 取樣演算法與 Shader Variant 控制

外描邊的繪製方法可由以下開關精細調整。這些是 Shader Variant 級別的設定，影響 GPU 編譯出的指令數量。Outline 為整顆 Shader 中最重的 Pass，手機平台建議於 §25 BuildConfig 中關閉。

Screen Space Width	啟用後 Outline Width 以螢幕空間像素計算（距離鏡頭遠近也保持等粗）；關閉則為模型空間。	_USE_SCREENSPACE_OUTLINE_WIDTH
Fill Inside	啟用後外描邊填入主紋理內部 — 受擊閃白/紅效果。若由程式 Runtime 控制，Preset 請取消勾選此項。	_OUTLINE_FILL_INSIDE
Sample 8 Neighbours	啟用 8 鄰域 Sobel（標準）；關閉則為 4 鄰域，速度較快但有鋸齒。	_USE8NEIGHBOURHOOD_ON
Reference Texture Width	Outline Width 的單位參考解析度，預設 1024。若主紋理 2048，須相應調整。	_OutlineReferenceTexWidth
Opaque Alpha Threshold	主紋理 Alpha 大於此值視為完全不透明 — 避免半透明區域產生雜訊邊緣。	_OutlineOpaqueAlpha · 0–1
Outline Mip Level	取樣 Mip Level (0–3)。Mip 越高描邊越柔但會抖動；常用 0。	_OutlineMipLevel

進階區塊（Directional Mask / Tint / Noise）以 _0L_ 前綴，結構同 §10.3。

§ 12 · SPINESHADOWLIGHT

自製光源元件 — Point · Spot · Area + Cookie

與 Unity Light 平行的自製光源系統。在 Hierarchy 中右鍵「GameObject / Light / Spine Shadow Light」即可建立。Inspector 由兩個可摺疊區塊組成：💡 Light Settings 與 🍪 Cookie Settings。Scene View 提供 Range / Spot Angle / Area Size 的拖曳 Handle。

💡 Light Settings

type	Point / Spot / Area — 全向、聚光、體積區域光
blendMode	Multiply (陰影層) / Add (純發光) / Screen (柔和提亮) / Overlay (高飽和對比)
color (HDR)	本體顏色。HDR 大於 1 觸發 Bloom；預設純黑表示「投射陰影」
intensity	Master 倍率 0–10
spineIntensity	針對 Spine / Sprite 的強度倍率
terrainIntensity	針對 Terrain 渲染的強度倍率
range	影響距離 — 若燈光無效請優先檢查此項
spotAngle	聚光張角 1–179°（僅 Spot 生效）
areaSize	區域光長寬 Vector2（僅 Area 生效；Z 軸由 range）
cullingMask	LayerMask — 預設 {8, 9, 12, 13, 18, 20}。若不包含角色 Layer 則無效

🍪 Cookie Settings

cookieMode	Normal（單層靜態投影） / Caustics（雙層錯位水波焦散）
cookieTexture	投影紋理。Caustics 模式會嘗試自動載入預設 T_Caustics_0100
colorChannel	RGBA / R / G / B / Alpha — 單通道萃取作為強度遮罩
cookieScale (X, Y)	投影縮放 — 值越小投出的圖案越大
autoScroll	啟用後 Offset 轉為 UV/秒 流動速度；Edit Mode 也運作
cookieOffset	未開 Auto Scroll 時為靜態 UV；開啟後為每秒流動速度 Vector2

限制 — 場景中同一角色最多同時受 8 盞 Shadow Light 影響。超過時 SpineLightManager 依距離排序，取最近 8 盞。

§ 13 · SPINEHITSWEEPEFFECT

受擊掃光控制元件

Material 上的 Enable Hit Sweep 勾選後，本元件會自動掛載到角色 GameObject。Inspector 含兩個區塊：Animation Settings（節奏控制）與 Debug & Test Tools（編輯器測試介面）。實際遊戲中由戰鬥邏輯呼叫 `PlaySweep(Vector3)`。

Animation Settings

duration	PlaySweep 呼叫後光帶掃完全身的時間（秒）
sweepRange	Shader 中光帶起點 X 到終點 Y 的座標範圍，預設 -4 → 4。角色巨大時請加大為 -8 → 8
sweepCenterOffset	抵銷 Spine 原點通常在腳底的問題。讓系統知道角色「胸口」位置

Debug & Test Tools

 取自身座標	將 debugHitPosition 設為角色 transform.position
在 Scene 編輯點位	於 Scene View 拖曳紅色 Handle 設定打擊點。同時顯示紅色虛線指向角色中心
▶ Test Sweep Effect	立即觸發 PlaySweep(debugHitPosition) 測試動畫

程式呼叫範例 — 戰鬥腳本

```
// 1. 取得角色身上的特效腳本
SpineHitSweepEffect hitEffect
    = GetComponent<SpineHitSweepEffect>();

if (hitEffect != null)
{
    // 2. 傳入受擊點的世界座標
    // 協程驅動 _SweepProgress 由 0 至 1
    hitEffect.PlaySweep(hitWorldPosition);
}
```

運作原理 — 元件透過 MaterialPropertyBlock 寫入 `_SweepProgress`、`_SweepHitPosOS`（本地座標）、`_SweepRange`、`_SweepCenterOffset`，不打斷 SRP Batching。協程結束後恢復 0，下次呼叫從頭開始。

§ 14 · SPINELIGHTMANAGER 場景單例

光照資料的中央交換站

SpineLightManager 為每個 Scene 一個單例。任一光照模組首次啟用時 EnsureInstance () 自動建立 GameObject — 無需手動新增。每幀以 LateUpdate 進行光源收集、排序、SH 取樣、MaterialPropertyBlock 注入。

資料 · Inputs

collectLightMask	LayerMask — 哪些 Layer 上的 Light 會被收集
targetLights	List<Light> — Unity Realtime Light 列表
targetShadowLights	List<SpineShadowLight> — 自製光源列表
targetCharacters	List<Renderer> — 已註冊接收的角色
probeSampleNormal	Vector3 — SH 取樣方向（預設 up）

流程 · Per Frame (LateUpdate)

- 01

收集 & 排序
以「光源 ↔ 角色」距離平方計算，取最近 8 盞放入 _SpineLightPos / Color / Spot 陣列。
- 02

SH 取樣
LightProbes.GetInterpolatedProbe → 9 個係數轉成 _SpineSHAr ~ _SpineSHC。
- 03

Cookie 綁定
為每盞 ShadowLight 設定 _SpineShadowCookie0–7 + UV 參數。
- 04

MaterialPropertyBlock
逐 Renderer 套用 SetPropertyBlock — 不污染共享材質球。

§ 15 · SPINELIGHTRECEIVER

角色端接收元件 + Active Lights Debug 面板

LightReceiver 是 LightManager 在角色端的對等元件。OnEnable 時呼叫 Manager.RegisterCharacter 完成註冊。Inspector 提供「🔍 Active Lights」唯讀面板，即時顯示當下打在這個角色身上的所有光源 — 對偵錯「燈光該打到卻沒打到」極為有用。

自動掛載條件

當以下任一勾選時系統會自動為角色掛載 LightReceiver：

- Enable Normal Map
- Enable BackLight（光照模式 ≠ Disabled）
- Enable Inline（光照模式 ≠ Disabled）
- Enable Outline（光照模式 ≠ Disabled）
- Enable Light Probe
- Enable Shadow Light Base

OnDisable 行為 — 元件被停用或角色被銷毀時主動清除所有殘留的 MaterialPropertyBlock 資料，避免「光照不消失」、「上一場景的光延續到新場景」等 bug。

🔍 Active Lights – Runtime Debug 面板

Realtime Lights

金黃色條 — 列出當下被本角色取用的 Unity Realtime Light 物件，可點擊跳轉

Shadow Lights

天藍色條 — 列出當下被本角色取用的 SpineShadowLight 物件

清單於 Edit Mode 與 Play Mode 都會每幀刷新。每個條目顯示為唯讀的 ObjectField，可直接點擊 Ping 場景中的對應物件。

清單為空但角色看起來受光不正常 — 多為角色 Layer 不在收集名單，或燈源 cullingMask 未包含角色 Layer。請啟用 §30 排查器。

§ 16 · 其他三個自動管理的元件

SpineAlphaMaskRenderer · SpineBloomThresholdSync · SpineEffectBootstrapper

這三個元件不會在角色 Inspector 上被直接操作 — 它們由 SpineCustomShaderGUI 在屬性變動時自動掛載 / 移除。本節說明各元件的職責、觸發條件、與重要欄位。

觸發：Enable Inline 勾選

SpineAlphaMaskRenderer

使用 CommandBuffer 在主相機 BeforeForwardOpaque 時繪製一張 RenderTexture（Alpha Mask）供 Inline Shader 取樣。

resolutionScale	RT 解析度倍率，預設 0.75
綁定相機	Camera.main，場景切換時自動重新綁定

觸發：Suppress Bloom 勾選

SpineBloomThresholdSync

透過 Reflection 抓取後製 Beautify 的 finalBloomIntensity，依 AnimationCurve 推算 _BloomSuppressThreshold 寫入 MPB。

驕標記	lastAppliedBloom 未變則跳過寫入
預設曲線	1.0 → 0.60；10.0 → 0.52

手動使用（程式 API）

SpineEffectBootstrapper

執行階段一次性套用一份 Profile，並自動補齊所有依賴元件（Receiver / AlphaMask / HitSweep / BloomSync）。適合角色切換特效時引入完整效果包。

ApplyProfile()	套用 + 補齊依賴
ClearAllEffects()	一次清除所有 Spine 特效

共同前提 — 自動掛載僅對 Layer ∈ {8, 9, 12, 13, 18, 20} 的物件作用。Validate 機制會在每次選取時掃描；若 Layer 不正確會把元件移除並在 Inspector 頂部跳出醒目警告。

§ 17 · PRESET 系統概念

把模組設定存成資產 — 跨材質球共用。

每個子模組（Main & Shadow、Inline、Outline 等）皆可獨立儲存為一個 ScriptableObject (SpineModuleProfile.asset)。掛載後 Inspector 中該模組的屬性會顯示 ◆ 符號，並由 SpineProfileAnimator 透過 MaterialPropertyBlock 即時驅動 — 不寫入材質球本體，多個角色可共用。

操作步驟（Inspector 端）

- 01

點 P 按鈕展開 Preset 面板

每個子模組標題右側皆有 P 圖示。
- 02

設定 Save Mask 後 Save

勾選想保存的屬性 — 未勾選的不會寫入 Preset。
- 03

自動防呆路由

將 Inline 的 Preset 拖到 Outline 區？系統會偵測 moduleName 並自動導向正確分區。
- 04

綁定持久化

Preset GUID 寫入 Material.SetOverrideTag — 重新打開 Inspector 也會記得。

關鍵設計原則	
儲存類型	Float / Color (HDR) / Vector4 / Texture
每個屬性	可選擇靜態值，或開啟 ~ 切到 AnimationCurve 驅動
寫入機制	SpineProfileAnimator 每幀更新 MaterialPropertyBlock，不污染材質
忽略名單	ProfileIgnoredProperties — Enable Toggle 與 LightingMode 不入 Preset，避免動畫切換模組
場景同步	SpineModuleProfileEditor.SyncAllRenderersUsingThisProfile — 詳見 §18

§ 18 · 直接點選 PROFILE 資產時的編輯介面

SpineModuleProfileEditor — 全場景自動同步

在 Project 視窗中直接點選一個 .asset 預設檔時，會由 SpineModuleProfileEditor 接管 Inspector。此編輯器最關鍵的特性 — 修改 Profile 的同時，會自動掃描全場景所有套用此 Profile 的角色，刷新自動掛載狀態。

編輯器特性

頂部色帶	依 moduleName 自動著色 — 一眼看出這份 Profile 是哪個分區
屬性切換	「顯示 Shader 變數名稱」Toggle，可在友善名稱與 _PropertyName 間切換
~ 曲線編輯	每個屬性右側可切換靜態值 / AnimationCurve 模式（含預設關鍵幀）
Color/Vector	展開後分別顯示 R/G/B/A 或 X/Y/Z/W 四條獨立曲線

屬性類型自動判斷：開頭為 _Enable / _Use / _Fill / _Invert 或含 Affects 顯示為 Toggle；含 LightingMode 顯示為 EnumFlagsField；含 ZTest 顯示為 EnumPopup。

SyncAllRenderersUsingThisProfile — 修改後自動同步

當你修改 Profile 的任一屬性，編輯器會：

- 01

掃描全場景 Renderer
FindObjectsByType<Renderer>()。
- 02

篩 Layer 與 Profile
Layer ∈ {8,9,12,13,18,20}，且 SpineProfileAnimator.profiles 含本 Profile。
- 03

Profile > Material 雙重驗證
先查 Profile 中是否定義該功能；無則查 Material — 決定是否需自動掛載。
- 04

SyncAutoComponent
補齊 AlphaMaskRenderer、HitSweepEffect、LightReceiver、BloomThresholdSync；若需光照則 EnsureSpineLightManager()。

§ 19 · SAVE MASK

參數遮罩 — 程式衝突的最佳解

優先權：**MaterialPropertyBlock**（最高） > Material Instance > Shared Material。

若某個參數需由程式在 Update 中「每幀高頻控制」（如閃爍、變色），請在 ShaderGUI 的 Save Properties 中**取消勾選**該參數。

如此 Profile 不會綁定該參數，程式可安全用 SpineProfileHelper.SetFloat 直接修改 MPB，不會被 Animator 覆寫。

常見錯誤 — 把 Effect Color 或 Fill Inside 等需要程式控制的屬性寫入 Preset，導致受擊閃白功能無效。

§ 20 · ~ ANIMATIONCURVE 驅動

逐屬性的曲線動畫引擎

每個 ◆ 屬性右側都有 ~ 按鈕。點開後該值改由 AnimationCurve 取代靜態值；時間軸從 GameObject 生命週期開始，可由 Animator.Restart() 重置。

Float	一條 curveX 控制 0–1 數值
Color	四條曲線 (R/G/B/A) 各自驅動
Vector4	四條曲線 (X/Y/Z/W) 各自驅動
Texture	不支援曲線（僅靜態值）

被忽略的屬性 — Profile 不儲存 Enable 類 Toggle 與 LightingMode（避免動畫意外切換整個模組）。詳見 ProfileIgnoredProperties 清單。

§ 21 · SPINENAMINGRULEWINDOW — 預設檔命名規則

兩種命名格式 — 角色特化與通用型。

所有 Preset 預設檔都應遵循下列其中一種命名格式。SpineNamingRuleWindow 為每個 Shader 模組提供範例與一鍵複製按鈕，避免拼錯字。

A · 角色特化功能

{角色英文代稱}_{模塊名稱}_{序號}

為單一角色設計、不會跨角色共用的特效。例：

Player_Outline_001 Boss_BackLight_001 NPC_Emission_002

序號從 001 起跳。預設為 Index — 請手動改為實際序號。

B · 通用型功能

General_{模塊名稱}_{用途}_{序號}

可被多角色重用的狀態效果。用途為實際效果簡稱，例：

General_Overlay_Toxic_001 General_Outline_Frozen_001 General_BackLight_Default_001

用途預設為 Default — 請改為實際效果簡稱（Toxic、Frozen、Burn 等）。

支援的模組名稱 — Main / Shadow、PostProcessing、GlobalNoise、NativeLighting、ShadowLight、NormalMap、LightProbe、Emission、EffectOverlay、HitSweep、BackLight、Inline、Outline。
NamingRuleWindow 內含複製按鈕。

§ 22 · 程式端動態 PROFILE 控制

SpineProfileHelper — 三個核心 API

強烈建議使用 SpineProfileHelper 靜態類別控制 Profile，而非自行操作 SpineProfileAnimator 元件。Helper 內建生命週期防錯、重複掛載偵測與效能自動回收。

三個高階 API（推薦）

```
// 1. 動態加載 / 替換 Profile
//    自動掛載 Animator、防重複、強制刷新
SpineProfileHelper.ApplyProfileRuntime(
    myRenderer, newProfile);

// 2. 移除指定模塊（Enum 避免拼錯）
SpineProfileHelper.RemoveProfileRuntime(
    myRenderer,
    SpineProfileModule.Outline);

// 3. 一鍵清除角色身上所有 Profile
//    與緩存
SpineProfileHelper.ClearAllProfiles(myRenderer);
```

SpineProfileModule Enum

取代字串避免拼錯：

.Main .PostProcessing .GlobalNoise .NativeLighting .ShadowLight .NormalMap
.LightProbe .Emission .EffectOverlay .HitSweep .BackLight .Inline
.Outline

輔助方法 — ToModuleName()、ToTagKey()、IsValidProfile() — 編譯時即可發現錯誤，不用等執行才知道 typo。

§ 23 · 進階 — 臨時副本竄改法

ApplyTempProfile — 狀態切換的最佳解

當角色進入特殊狀態（中毒、冰凍、變身）時，**不建議**直接修改 MPB — 因為會被曲線蓋掉。最佳解：呼叫 ApplyTempProfile，系統會複製一份 Profile，將指定參數強制覆寫，並關閉該參數的曲線動畫。

```
// 1. 準備參數字典
var overrides = new Dictionary<string, object> {
    { "_OutlineColor", Color.green },
    { "_OutlineWidth", 5.0f }
};

// 2. 套用副本，回傳實體存起來
SpineModuleProfile activeTempProfile
    = SpineProfileHelper.ApplyTempProfile(
        myRenderer, baseProfile, overrides);

// 3. 狀態結束時，移除效果並銷毀副本
SpineProfileHelper.RemoveProfileRuntime(
    myRenderer, activeTempProfile.moduleName);
Destroy(activeTempProfile);
```

記憶體洩漏警告 — ApplyTempProfile 回傳的 TempProfile 實體**必須手動 Destroy**。否則每次狀態切換都會在記憶體中產生新的 ScriptableObject 殘留。

底層原理 — Temp Profile 為原 Profile 的深拷貝，在 overrides 中指定的屬性會被覆寫且 useCurve = false（停用曲線）。原 Profile 不會被修改。

典型使用場景 — 受擊瞬間將 Outline 變紅、變身時將整個角色 Emission 染色、無敵狀態下將描邊閃光放大 — 任何「臨時改某個 Profile 的某幾個屬性」的場景。

§ 24 · 底層 MATERIALPROPERTYBLOCK 控制

當 Save Mask 中的屬性未被勾選 — 用這組 API。

若某個參數在 Save Mask 中未被勾選（即 Profile 不會驅動該屬性），程式可使用 Helper 封裝的靜態 API 安全寫入 MPB — 與系統底層的 MPB 狀態無縫融合，不打架，並保持極致合批效能。

✓ 正確寫法

```
// 寫入 Float
SpineProfileHelper.SetFloat(myRenderer,
    "_OutlineWidth", 0.1f);

// 寫入 Color
SpineProfileHelper.SetColor(myRenderer,
    "_EffectColor", Color.red);

// 寫入 Vector4
SpineProfileHelper.SetVector(myRenderer,
    "_GlobalNoiseSpeed",
    new Vector4(1, 1, 0, 0));
```

無材質實例化、不污染 sharedMaterial、不打斷 SRP Batching。

✗ 錯誤寫法

```
// x 會產生材質實例
// x 可能被 Animator 覆寫
// x 打斷 SRP Batching
myRenderer.material.SetFloat(
    "_OutlineWidth", 0.1f);

// x 也是錯的
myRenderer.sharedMaterial.SetFloat(
    "_OutlineWidth", 0.1f);
// 修改所有共享此材質的角色！
```

前提 — 必須確保該參數在 ShaderGUI 儲存 Preset 時沒被勾選。否則 Animator 會把你寫入的值覆蓋掉。

§ 24.1 · 互動式測試工具

SpineProfileTester — 滑鼠左鍵 / 右鍵 / 中鍵測試

在 Hierarchy 內加入此測試元件後進入 Play Mode，可用滑鼠快速驗證 Profile 系統的完整流程。建議在新建 Profile 後使用此測試器確認運作正確再正式接入專案。

滑鼠左鍵

套用 / 移除 Profile

在指定 Renderer 上動態套用 / 移除指定的 Profile 資產。對應呼叫：ApplyProfileRuntime 與 RemoveProfileRuntime。

滑鼠右鍵

切換 MPB 數值

直接修改指定 _PropertyName 的 MPB 數值（不經 Profile），用以驗證 Save Mask 未勾選屬性的程式控制是否正常。

滑鼠中鍵

產生臨時 Profile

呼叫 ApplyTempProfile 產生竄改副本，用以驗證 §23 的臨時覆寫流程。

建議流程 — 新建 Profile 後 → 拖入測試器 → Play → 左鍵套用 → 右鍵竄改未勾選屬性 → 中鍵測試臨時 Profile → 三項都正常即可正式接入專案戰鬥邏輯。

§ 25 · 平台變體管理中心 EDITORWINDOW

SpinePlatformManagerWindow — 滑動式平台切換器

Window 選單可開啟「Spine 平台變體管理」視窗。視窗以滑動卡片方式列出 Editor Default 與所有平台 Config (iOS / Android / Windows / Switch / PS5 / Xbox / WebGL …)。每張卡片顯示該平台啟用的 Shader Feature 開關。

操作機制

滑動切換	左右拖曳卡片選擇平台。卡片中心放大，遠端淡出。
即時開關	點擊功能列即時切換該模組的啟用狀態 — 設有滑動防誤觸機制。
套用按鈕	底部「套用 {平台名}」按鈕 — 一鍵刷新全專案 Spine 材質球的 Shader Keyword。
頂部狀態	顯示「當前套用：{平台名}」與動態色彩漸變橫條。
+ 新建	右上 + 按鈕建立新的 ShaderConfig_{Platform}.asset。
🗑 移除	非 Editor 卡片可按移除鍵刪除該平台 Config 資產。

每個平台卡片內含三組區塊

LIGHTING MODULES

Native Lighting · Shadow Light Base · Light Probe · Normal Map

VFX MODULES

BackLight · Emission · Inline (Rim) · Outline · Hit Sweep · Effect Overlay

GLOBAL SETTINGS

Global Noise ； Outline 啟用時展開 𠂇 8 Neighbourhood · 𠂇 Screen Space Width · 𠂇 Fill Inside

「? 指南」按鈕 — 視窗右上角的指南按鈕會打開重疊式說明面板，內含程式 API 範例 (SpineShaderBuildPipeline.SyncKeywordsForTarget) 與一鍵複製、一鍵跳轉設定檔目錄的按鈕。

§ 26 · SPINESHADERBUILDCONFIG + 雙重驗證

Editor Default 與 Platform Config 的差異。

系統提供兩種運作模式 — 編輯器中所有 Keyword 強制全開以供美術預覽（Editor Default）；切換到平台 Config 時則以「雙重鎖」決定材質球的 Keyword 啟用狀態。

A · Editor Default 模式

執行 SpineShaderBuildPipeline.SyncAllEnabled() 後，會強制掃描全專案的 Spine 材質球，將所有 15 個 Module Keyword 一律 EnableKeyword。SessionState 記為 EditorMode = true。

適合美術做設計時 — 不論勾不勾，都能看到效果。**不影響正式打包效能。**

B · Platform Config 模式

執行 SyncForPlatformConfig() 後改用雙重驗證：

isAllowedByConfig	該 Keyword 在平台 Config 中是否被勾選
isToggledByUser	材質球中對應 _Enable* 屬性是否為 ON
最終結果	只有「兩者都成立」時才 EnableKeyword

第三道過濾 — OnProcessShader（打包時） — IPreprocessShaders 介面在編譯每個 Variant 時，會檢查兩件事：(1) 平台 Config 是否放行此 Keyword；(2) 全專案是否有任一材質球實際用到此 Keyword。任一不符即將該 Variant 從 IList<ShaderCompilerData> 中移除。Outline / Inline 等重 Pass 在不需要時**完全不會編譯**。

§ 27 · SPINESHADERBUILDPipeline 程式 API

為打包自動化腳本使用。

若你的專案有自動化打包流程（CI / 一鍵打包按鈕），可透過 SpineShaderBuildPipeline 的靜態方法在 OnPreprocessBuild 階段刷新材質球設定。

```
public class SpineBuildAutomation {
    public void PrepareForTarget() {
        // 傳入名稱，刷新全專案材質球
        // 例如切到 Android
        SpineShaderBuildPipeline
            .SyncKeywordsForTarget("Android");
    }
}
```

命令式 API — 無需開啟 EditorWindow。可在 CI 中執行。

完整 API 清單

SyncAllEnabled()	恢復 Editor Default 全效預覽模式
SyncForPlatformConfig(cfg)	依指定 Config 套用平台雙重鎖
SyncKeywordsForTarget(name)	依平台名稱（字串）切換 — CI 用
IsEditorDefaultMode()	查詢當前是否為 Editor Default 模式
OnPreprocessBuild	自動於打包前掃描，按 BuildTarget 套用對應 Config

關於 **MissingReferenceException** — 若 Config 資產被外部刪除或遺失，PlatformManager 視窗會偵測並顯示「 發生異常 (MissingReferenceException)」面板，提供「重新整理」按鈕讓你恢復視圖。

§ 28 · LAYER 規約 — 所有自動化的基礎

角色必須在六個 Layer 之一。

SpineLightManager 的 collectLightMask、SpineShadowLight 的 cullingMask、SpineCustomShaderGUI 的 Validate 機制 — 全部都鎖定在下列六個 Layer。Layer 不正確時，自動化機制會在每次選取角色時將 SpineAlphaMaskRenderer / SpineHitSweepEffect / SpineLightReceiver / SpineBloomThresholdSync 全部移除。

8

主角區

9

敵人區

12

NPC 區

13

道具區

18

特效層

20

UI 角色

違規後果 — Inspector 頂部會跳出醒目警告「⚠ 此角色 Layer 不在自動化白名單」與一鍵修正按鈕。光照與描邊都會失效。請點擊提示一鍵修正。

§ 29 · 光照問題自動排查器

SpineLightingIssueTroubleshooter — 八項自動檢測

選取場景中的角色 GameObject，於 SpineDebugHelpWindow 中按「光照問題排查」按鈕。工具會檢測八項常見問題，並提供「點此修正」與物件 Ping 跳轉。

01

SpineLightManager 存在

若無提供建立按鈕

02

Mask 含角色 Layer

collectLightMask 包含此 Layer

03

材質球 Shader 正確

Spine/Skeleton + Native Lighting ON

04

Layer 在白名單

8 · 9 · 12 · 13 · 18 · 20

05

至少一盞 Realtime

cullingMask 與角色 Layer 交集

06

Directional Light 存在

否則無方向光著色

07

Keyword 已啟用

Build Config 未剝離

08

Receiver 已掛載

否則 Manager 不推送資料

輸出格式 — 每條失敗結果都附帶「調整方式」、紅色高亮關鍵字（如「沒有」「找不到」「不在」），以及可點擊的 Ping 連結直接跳到問題物件。沒問題的項目以綠色高亮（「有」「是」「已」「正確」「完成」）。

§ 30 · SPINEDEBUGHELPWINDOW — 常見問題

兩個最常出現的問題與標準解法。

Q1 · 光照與陰影問題

「為何角色看起來黑黑的，明明有開燈但沒受光照？」

標準排查順序（DebugHelpWindow 列出）：

1. SpineLightManager 層級規範 — 該 Layer 是否被加入收集名單
2. 確認材質球已勾選 Enable Native Lighting
3. 確認 GameObject Layer $\in \{8, 9, 12, 13, 18, 20\}$
4. 確認場景燈光的 Culling Mask 包含上述 Layer
5. 按  全效預覽按鈕 — 若恢復正常，代表是平台 Config 剝離了該模組

Q2 · 特效與模組缺失

「為何勾選內外框、背光、掃光都不起作用？」

標準解法：

1. 再次確認 Layer $\in \{8, 9, 12, 13, 18, 20\}$ — 許多特效依賴腳本抓取資料
2. 按  全效預覽 — 強制開啟此材質的所有 Shader 變體
3. 若全效預覽下恢復正常，代表「平台變體管理」剝離了該 Keyword

本地完整預覽 — 全效預覽僅影響當前編輯器，不影響打包效能。隨時可開關。

§ 31 · 從零建立一個受光照的 SPINE 角色

六個步驟。

本節為最短上手流程。任一步驟出問題請翻閱 §29 排查器章節。

- 01

建器材質球，Shader 選 Spine / Skeleton
拖到 SpriteRenderer — 主紋理、Straight Alpha、_Normal / _BackLight / _Emission 系列檔自動就位。
- 02

角色 GameObject 設到 Layer 8（或 9 / 12 / 13 / 18 / 20）
否則自動化機制會清除所有掛載元件。Inspector 頂部有一鍵修正按鈕。
- 03

開啟需要的模組 — 例如 LIGHTING / Normal Map / Outline
SpineLightReceiver、AlphaMaskRenderer、LightManager 會在勾下 Toggle 時自動掛載。
- 04

在場景放一盞 Realtime Directional Light
確保其 Culling Mask 包含角色 Layer。可加入 SpineShadowLight 做特殊光罩或 Caustics 波光。
- 05

需要動畫？將模組存成 Preset 並加入 ~ 曲線
SpineProfileAnimator 即時驅動。命名請遵循 §21 規則。
- 06

打包前檢查 — 開啟 §25 SpinePlatformManagerWindow，切到目標平台
確認該平台 Config 中啟用了你需要的所有 Module Keyword，按「套用 {平台名}」刷新材質球。

§ 32 · 文件結束 — 取得協助

三個取得協助的管道。

本文件描述的所有功能於 Inspector 中皆可直接操作 — 大多數情境不需閱讀程式碼。若遇到本文件未涵蓋之情況，請依下列順序處理。

A · 線上文件 右鍵任一參數 「 查看使用說明」自動跳轉至對應 Help Window 並 Ping 高亮。	B · 視覺檢查  全效預覽 / Reset 切換平台模擬比對效果，或重置整顆材質。	C · 系統檢查 SpineLightingIssueTroubleshooter 逐項自動診斷 8 個常見光照問題 + 一鍵修正。
命名空間	Shader 路徑	文件版本
VFXTool.SpineSkeletonShaderTool	Spine / Skeleton	Revision 2026.01

Spine Skeleton Shader · 11 個 Runtime 腳本 · 7 個 EditorWindow · 1 套 Shader · 1 條打包流水線 · 1 份說明書。